



ASSIGNMENT REPORT

Digital Design

Assignment 4.1 — Verilog Design of 8×16 Register File

Course	Digital Design
Assignment	Assignment 4.1 – Verilog Design of 8×16 Register File
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	]Eng. Ali Temsah
Submission Date	[Date]

01 OBJECTIVE

The objective of this assignment is to design, in Verilog, an 8×16 register file — a bank of eight 16-bit registers sharing a common address, write-data, and read-data bus — and to verify its read, write, and asynchronous-reset behavior through a self-checking testbench simulated in ModelSim.

02 PROBLEM STATEMENT / REQUIREMENTS

The register file must provide 8 addressable registers, each 16 bits wide, accessed through a single shared address bus used for both read and write operations. Only one operation (read or write) is evaluated at a time, and both are synchronized to the positive edge of the clock. All registers must be clearable through an asynchronous, active-low reset. The specific requirements are:

- 8 registers, each 16 bits wide (Register_File_MEM[0:7]).
- Single shared 3-bit Address bus ($\log_2(8) = 3$) used for both read and write access.
- 16-bit WrData input bus and 16-bit RdData output bus.
- Write Operation executes only when WrEn = 1, on the positive edge of CLK.
- Read Operation executes only when RdEn = 1, on the positive edge of CLK.
- Only one of WrEn / RdEn is evaluated per clock edge (read and write are mutually exclusive).
- Asynchronous, active-low Reset (RST) clears all 8 registers to 16'h0000, independent of CLK.

03 DESIGN & ANALYSIS

3.1 Design Approach

Since the register file holds 8 words, the Address bus is sized to $\log_2(8) = 3$ bits, wide enough to index registers 0 through 7. Each register, and both the WrData and RdData buses, are 16 bits wide as specified. The 8 registers are modeled as a Verilog memory array, Register_File_MEM[0:7], each element 16 bits wide.

A single always block, sensitive to the positive edge of CLK and the negedge of RST (to model the asynchronous, active-low reset per the assignment hint), implements the behavior. The reset condition is checked first and given top priority: whenever RST is low, all 8 registers are cleared on the same edge that triggered the block, regardless of CLK. When RST is high, the block instead evaluates on the rising edge of CLK: if WrEn is asserted, the word on WrData is written into Register_File_MEM[Address]; otherwise, if RdEn is asserted, the contents of Register_File_MEM[Address] are latched onto RdData. Because the two conditions are chained with else if, read and write can never occur on the same clock edge, matching the 'only one operation at a time' requirement.

3.2 Truth Table / State Table

The next-state behavior of the register file reduces to the following priority table, evaluated on every relevant edge:

RST	WrEn	RdEn	Operation
0	X	X	Asynchronous clear — all 8 registers $\leftarrow$ 16'h0000
1	1	X	Register_File_MEM[Address] $\leftarrow$ WrData (on posedge CLK)
1	0	1	RdData $\leftarrow$ Register_File_MEM[Address] (on posedge CLK)
1	0	0	Hold — no register or RdData update

Figure 1: Priority table for reset, write, and read operations.

3.3 Circuit / Block Diagram

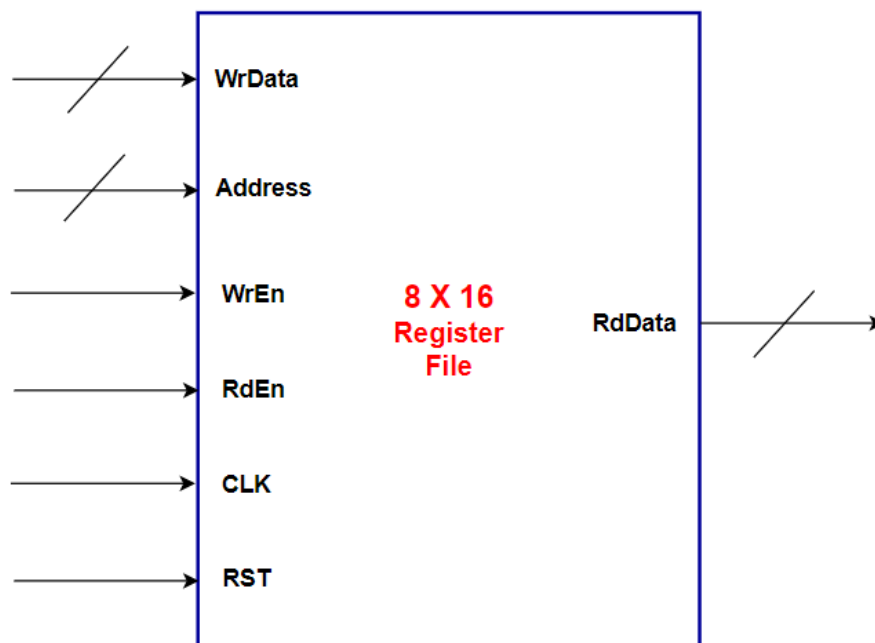


Figure 2: Block interface of the 8×16 Register File — WrData[15:0], Address[2:0], WrEn, RdEn, CLK, RST as inputs; RdData[15:0] as output.

3.4 HDL Implementation

The design was implemented as a single Verilog module, Register_File, with the reset branch given the highest priority inside the clocked always block, followed by the write branch, then the read branch:

```
module Register_File (
    input [15:0] WrData,
    input [2:0] Address,
    input WrEn,RdEn,
    input CLK,RST,
    output reg [15:0] RdData
);
    reg [15:0] Register_File_MEM [0:7];
    always @(posedge CLK or negedge RST) begin
        if (!RST) begin
            Register_File_MEM[0] <= 0;
            Register_File_MEM[1] <= 0;
            Register_File_MEM[2] <= 0;
            Register_File_MEM[3] <= 0;
            Register_File_MEM[4] <= 0;
            Register_File_MEM[5] <= 0;
            Register_File_MEM[6] <= 0;
            Register_File_MEM[7] <= 0;
        end
        else if (WrEn) begin
            Register_File_MEM[Address] <= WrData;
        end
        else if (RdEn) begin
            RdData <= Register_File_MEM[Address];
        end
    end
endmodule //Register_File
```

04 SIMULATION & RESULTS

A self-checking testbench, `Register_File_tb`, instantiates the `Register_File` as the unit under test and drives it with a 10 ns-period clock (100 MHz). It exercises six scenarios in sequence: an asynchronous reset, two write operations (to Register 0 and Register 3), two read operations (from Register 0 and Register 3), and a final reset to confirm that all registers are cleared again. Expected values are printed alongside simulated values via `$display`, and a `$monitor` statement logs every signal transition for cross-checking against the ModelSim waveforms below.

```
`timescale 1ns/1ps

module Register_File_tb;

    // Testbench signals
    reg [15:0] WrData;
    reg [2:0] Address;
    reg WrEn, RdEn;
    reg CLK, RST;
    wire [15:0] RdData;

    // Instantiate the Unit Under Test (UUT)
    Register_File uut (
        .WrData(WrData),
        .Address(Address),
        .WrEn(WrEn),
        .RdEn(RdEn),
        .CLK(CLK),
        .RST(RST),
        .RdData(RdData)
    );

    // Clock generation: 10ns period (100MHz)
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end

    // Test scenarios
    initial begin
        ... (see attached Register_File_tb.v for scenarios 3-6 and monitor/dump blocks)
    end
endmodule
```

4.1 Test Case Summary

Scenario	Stimulus Applied	Expected Result	Simulated Result	Status
1. Asynchronous Reset (0–20 ns)	RST = 0 for 20 ns, then RST = 1	All 8 registers cleared to 16'h0000 immediately (asynchronous, no clock edge required)	Register_File_MEM[0..7] = 0000 while RST = 0; contents remain 0 after RST deasserted	Pass
2. Write to Register 0 (20–50 ns)	Addr = 000, WrData = 0xABCD, WrEn = 1, one posedge CLK, then WrEn = 0	Register_File_MEM[0] loaded with 0xABCD on the clock edge	Register_File_MEM[0] = 101010111001101 (0xABCD); all other registers unchanged (0x0000)	Pass
3. Write to Register 3 (50–70 ns)	Addr = 011, WrData = 0x1234, WrEn = 1, one posedge CLK, then WrEn = 0	Register_File_MEM[3] loaded with 0x1234 on the clock edge	Register_File_MEM[3] = 0001001000110100 (0x1234); Register 0 still holds 0xABCD	Pass

4. Read from Register 0 (70–90 ns)	Addr = 000, RdEn = 1, one posedge CLK	RdData = 0xABCD (previously written value)	RdData = 1010101111001101 (0xABCD)	Pass
5. Read from Register 3 (90–110 ns)	Addr = 011, RdEn = 1, one posedge CLK	RdData = 0x1234 (previously written value)	RdData = 0001001000110100 (0x1234)	Pass
6. Verify Reset Clears All (110–140 ns)	RST = 0 for 15 ns, then RST = 1; read Addr = 000	All registers cleared; RdData = 0x0000 on subsequent read of Register 0	Register_File_MEM[0..7] = 0000; RdData = 0000000000000000	Pass

4.2 Simulation Waveforms

ModelSim waveform captures for each of the six testbench scenarios are shown below.

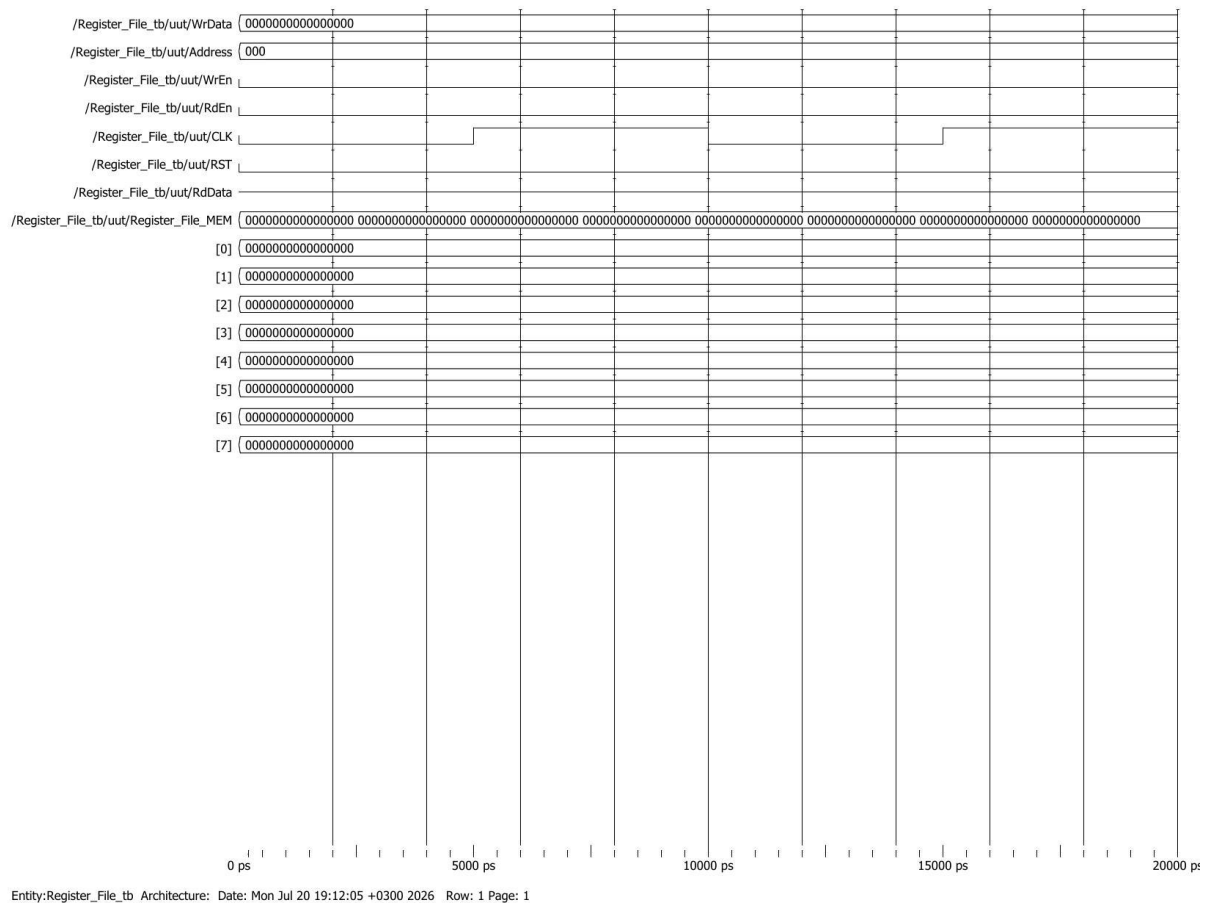
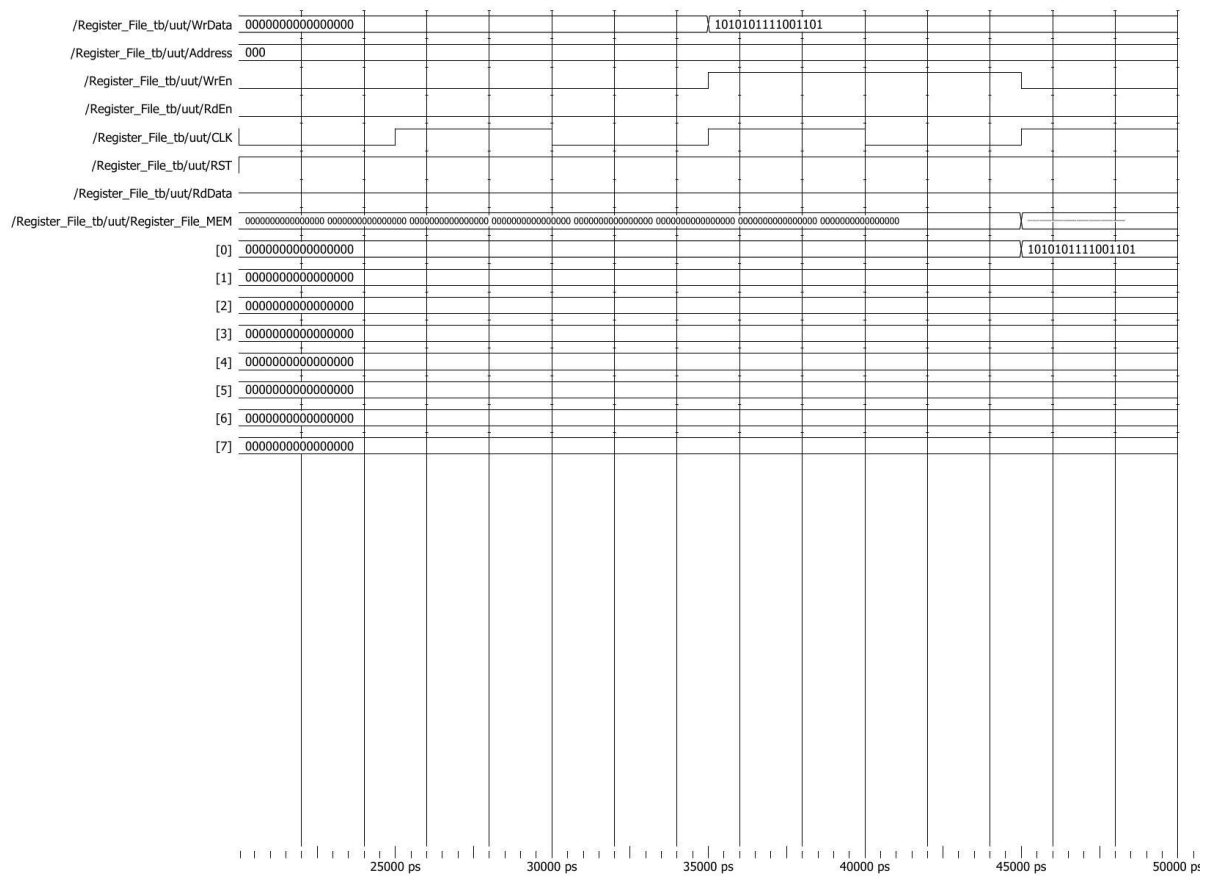
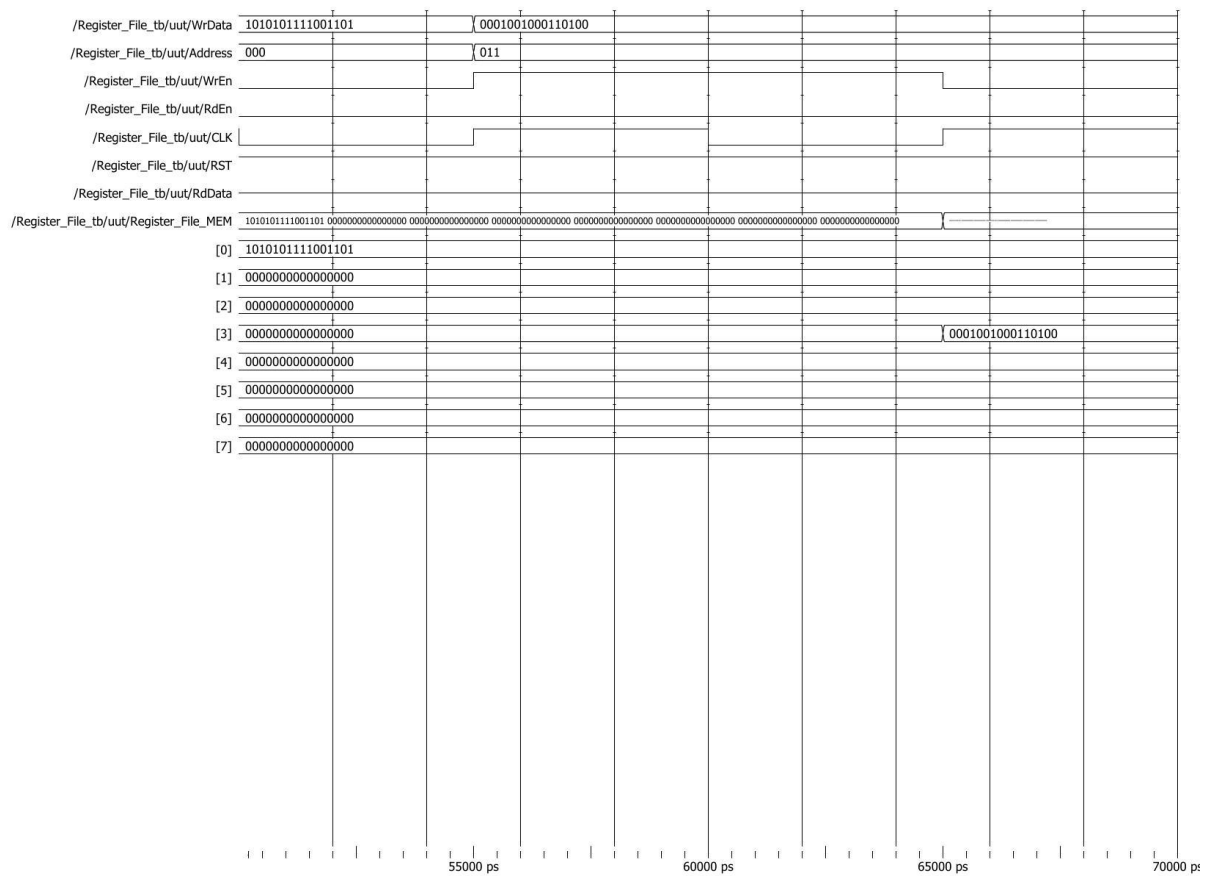


Figure 3: Scenario 1 — Asynchronous Reset (0–20 ns). RST held low clears Register_File_MEM[0..7] to 0x0000 independent of CLK.



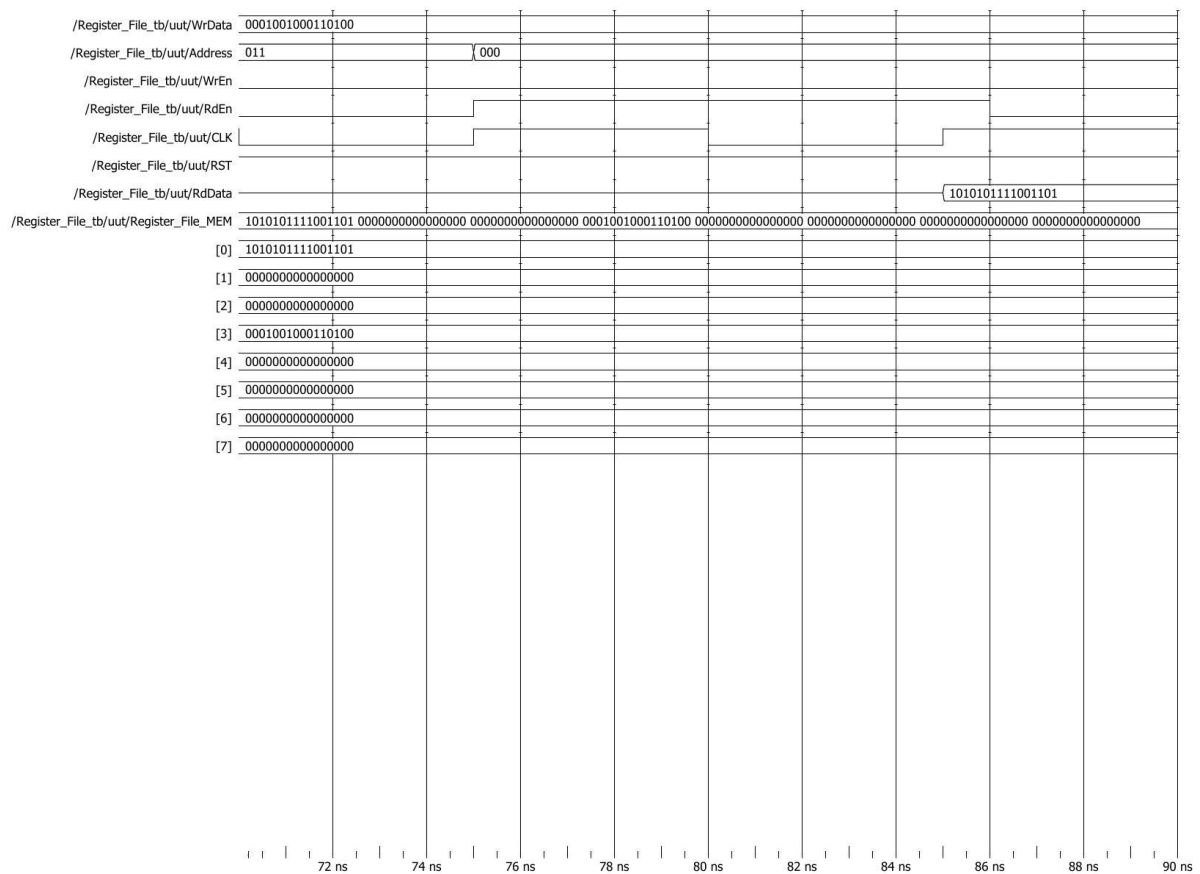
Entity: Register_File_tb Architecture: Date: Mon Jul 20 19:13:20 +0300 2026 Row: 1 Page: 1

Figure 4: Scenario 2 — Write to Register 0 (20–50 ns). $WrEn = 1$, Address = 000, $WrData = 0xABCD$; $Register_File_MEM[0]$ updates to 0xABCD on the clock edge.



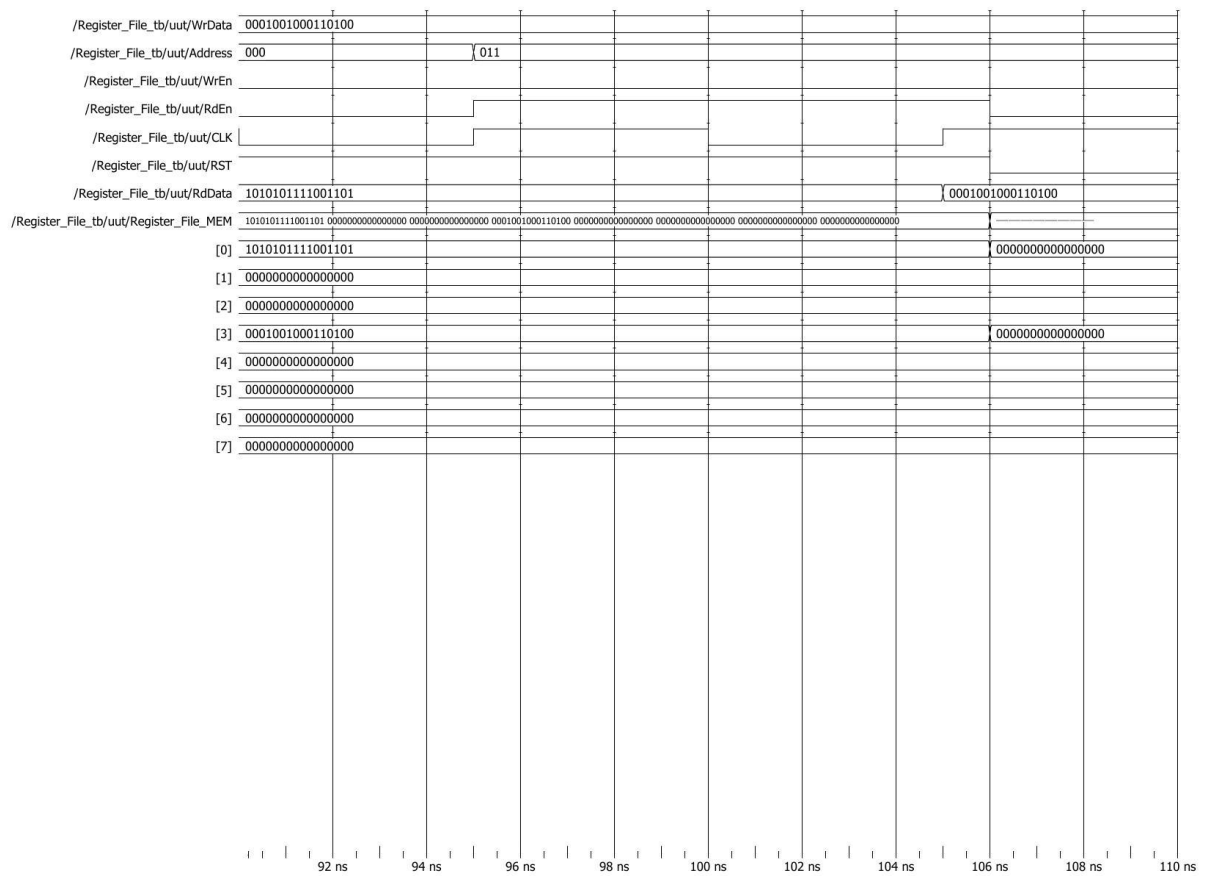
Entity: Register_File_tb Architecture: Date: Mon Jul 20 19:17:58 +0300 2026 Row: 1 Page: 1

Figure 5: Scenario 3 — Write to Register 3 (50–70 ns). $WrEn = 1$, Address = 011, $WrData = 0x1234$; Register_File_MEM[3] updates to 0x1234, Register 0 retains 0xABCD.



Entity: Register_File_tb Architecture: Date: Mon Jul 20 19:27:42 +0300 2026 Row: 1 Page: 1

Figure 6: Scenario 4 — Read from Register 0 (70–90 ns). RdEn = 1, Address = 000; RdData outputs 0xABCD.



Entity: Register_File_tb Architecture: Date: Mon Jul 20 19:29:02 +0300 2026 Row: 1 Page: 1

Figure 7: Scenario 5 — Read from Register 3 (90–110 ns). RdEn = 1, Address = 011; RdData outputs 0x1234.

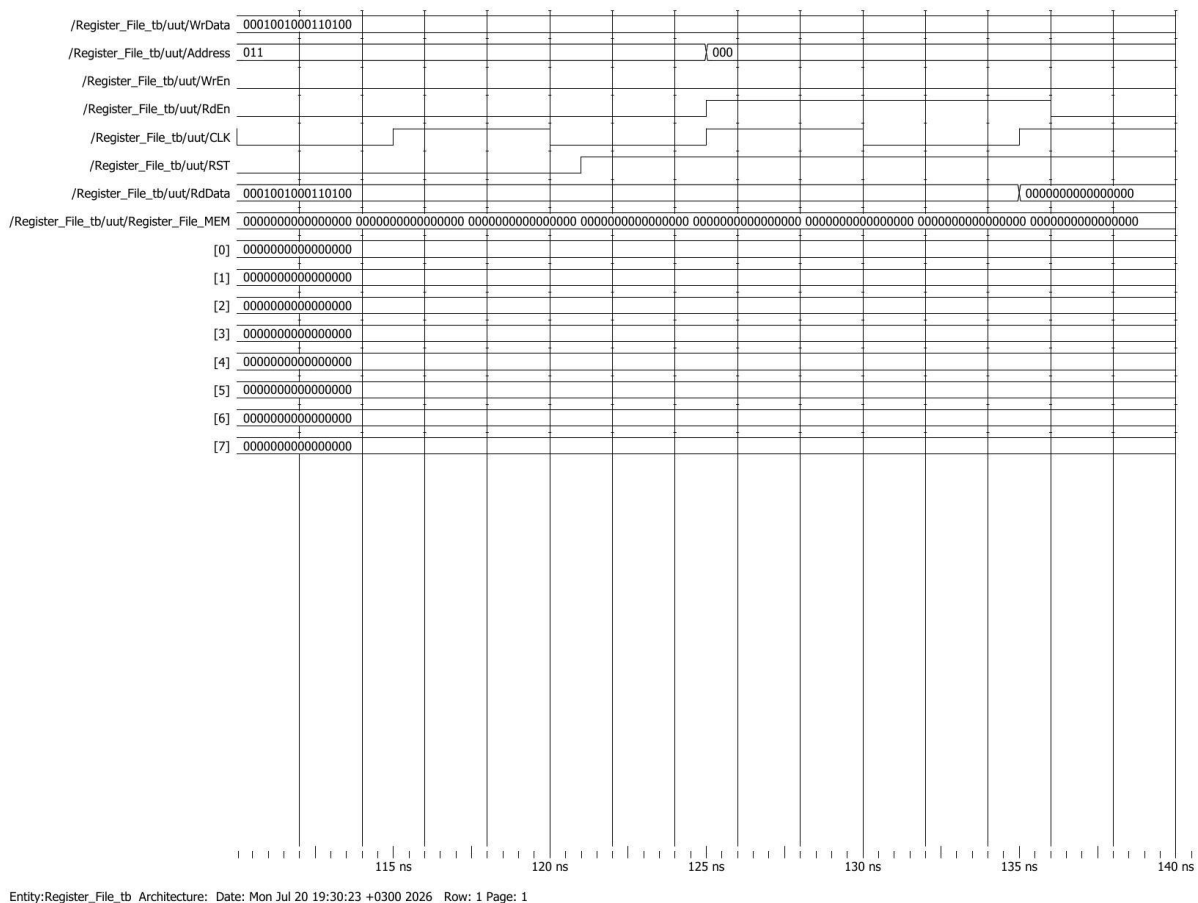


Figure 8: Scenario 6 — Verify Reset Clears All (110–140 ns). RST reasserted; all registers return to 0x0000, confirmed by reading Register 0 back as 0x0000.

4.3 Console / Transcript Log

The full ModelSim simulation transcript, produced by the \$display and \$monitor statements in the testbench, is reproduced below. It confirms, in text form, the same six scenarios shown in the waveforms above.

```
# =====
#   Register File Testbench Started
# =====
#
# --- Scenario 1: Asynchronous Reset ---
# Time=0 | RST=0 | WrEn=0 | RdEn=0 | Addr=0 | WrData=0x0000 | RdData=0xxxxx
# Time=20000 | RST=1 | WrEn=0 | RdEn=0 | Addr=0 | WrData=0x0000 | RdData=0xxxxx
# Reset completed. All registers cleared.
#
# --- Scenario 2: Write to Register 0 ---
# Time=35000 | RST=1 | WrEn=1 | RdEn=0 | Addr=0 | WrData=0xabcd | RdData=0xxxxx
# Time=45000 | RST=1 | WrEn=0 | RdEn=0 | Addr=0 | WrData=0xabcd | RdData=0xxxxx
# Wrote 0xabcd to Register 0
#
# --- Scenario 3: Write to Register 3 ---
# Time=55000 | RST=1 | WrEn=1 | RdEn=0 | Addr=3 | WrData=0x1234 | RdData=0xxxxx
# Time=65000 | RST=1 | WrEn=0 | RdEn=0 | Addr=3 | WrData=0x1234 | RdData=0xxxxx
# Wrote 0x1234 to Register 3
#
# --- Scenario 4: Read from Register 0 ---
# Time=75000 | RST=1 | WrEn=0 | RdEn=1 | Addr=0 | WrData=0x1234 | RdData=0xxxxx
```

```

# Time=85000 | RST=1 | WrEn=0 | RdEn=1 | Addr=0 | WrData=0x1234 | RdData=0xabcd
# Read from Register 0: 0xabcd (Expected: 0xABCD)
#
# --- Scenario 5: Read from Register 3 ---
# Time=86000 | RST=1 | WrEn=0 | RdEn=0 | Addr=0 | WrData=0x1234 | RdData=0xabcd
# Time=95000 | RST=1 | WrEn=0 | RdEn=1 | Addr=3 | WrData=0x1234 | RdData=0xabcd
# Time=105000 | RST=1 | WrEn=0 | RdEn=1 | Addr=3 | WrData=0x1234 | RdData=0x1234
# Read from Register 3: 0x1234 (Expected: 0x1234)
#
# --- Scenario 6: Verify Reset Clears All ---
# Time=106000 | RST=0 | WrEn=0 | RdEn=0 | Addr=3 | WrData=0x1234 | RdData=0x1234
# Time=121000 | RST=1 | WrEn=0 | RdEn=0 | Addr=3 | WrData=0x1234 | RdData=0x1234
# Time=125000 | RST=1 | WrEn=0 | RdEn=1 | Addr=0 | WrData=0x1234 | RdData=0x1234
# Time=135000 | RST=1 | WrEn=0 | RdEn=1 | Addr=0 | WrData=0x1234 | RdData=0x0000
# Read Register 0 after reset: 0x0000 (Expected: 0x0000)
# Time=136000 | RST=1 | WrEn=0 | RdEn=0 | Addr=0 | WrData=0x1234 | RdData=0x0000
#
# =====
#       Register File Testbench Finished
# =====
# ** Note: $finish      : Assignment 4/4.1/Register_File_tb.v(134)
#       Time: 156 ns  Iteration: 0  Instance: /Register_File_tb

```

05 DISCUSSION

All six scenarios matched their expected results exactly. The asynchronous reset cleared every register the moment RST went low, without waiting for a clock edge, confirming the negedge RST sensitivity in the always block behaves as an async, active-low reset rather than a synchronous one. The write scenarios confirmed that data is only committed to Register_File_MEM on a rising CLK edge while WrEn is high, and that writing one register (e.g., Register 3) does not disturb the contents of another (Register 0 correctly retained 0xABCD). The read scenarios confirmed that RdData correctly reflects the addressed register's contents one clock edge after RdEn is asserted, and the final reset scenario confirmed the register file can be cleared again mid-simulation, returning Register 0 to 0x0000. Because the always block chains the write and read branches with else if, no case arose where both a write and a read were evaluated on the same edge, satisfying the 'only one operation at a time' requirement. No discrepancies or timing violations were observed.

06 CONCLUSION

An 8 × 16 register file was successfully designed in Verilog with a shared 3-bit address bus, a 16-bit write-data bus, a 16-bit read-data bus, positive-edge-triggered read/write operations gated by RdEn/WrEn, and an asynchronous active-low reset. A six-scenario testbench — covering reset, two independent writes, two independent reads, and a second reset — was simulated in ModelSim, and every scenario produced the expected result, verifying correct functional behavior of the design.